

KoopCast++: Neighbour-aware Koopman Pedestrian Trajectory Prediction

Design specification & v1 results

June 22, 2026

Abstract. KoopCast++ predicts pedestrian futures with a *linear Koopman operator* acting on a lifted observable that already contains the (time-delayed) state, so *no decoder is needed*: the predicted positions appear directly in the observable and are read off by a fixed projection. The only learned component is a *social (neighbour) encoder*. The operator is obtained in closed form by *Extended DMD*, and the encoder is trained with a plain one-step Koopman *consistency* loss. The obstacle/environment branch of the earlier KoopyAdapt is dropped. Models are trained per held-out scene on the official ETH/UCY leave-one-out splits (Social-STGCNN distribution).

1 Setup

A scene has pedestrians $i = 1, \dots, N$ with positions $\mathbf{p}_i^t \in \mathbb{R}^2$ (world metres). Observation length $T_o=8$, prediction length $T_p=12$, $\Delta t=0.4$ s. At the prediction origin t_0 we observe $X_i = [\mathbf{p}_i^{t_0-T_o+1}, \dots, \mathbf{p}_i^{t_0}]$ and predict $[\mathbf{p}_i^{t_0+1}, \dots, \mathbf{p}_i^{t_0+T_p}]$.

2 Observable (lifted state)

The observable stacks a **time-delay history** of the state with a social embedding ($d = 2T_o + d_s = 16 + d_s$):

$$\boldsymbol{\psi}_i^t = \left[\underbrace{\mathbf{p}_i^t, \mathbf{p}_i^{t-1}, \dots, \mathbf{p}_i^{t-7}}_{16 \text{ (time-delay)}}; \underbrace{\mathbf{e}_i^t}_{d_s \text{ social}} \right] \in \mathbb{R}^d.$$

Because the raw positions are coordinates of $\boldsymbol{\psi}$, advancing $\boldsymbol{\psi}$ moves the most-recent-position slot to the next step; the predicted position is recovered by a *fixed* read-out $C = [I_2 \ \mathbf{0}]$, i.e. $\hat{\mathbf{p}} = C\boldsymbol{\psi}$. **There is no learned decoder.**

Social encoding. Around ego i at time t , neighbours within radius $R = 2.0$ m are binned into a $S=12$ -sector polar proximity histogram $\mathbf{g}_i^t \in \mathbb{R}^S$,

$$\mathbf{g}_i^t[k] = \max_{j: \angle(j) \in \text{sec } k, d_{ij} \leq R} \left(1 - \frac{d_{ij}}{R} \right), \quad d_{ij} = \|\mathbf{p}_j^t - \mathbf{p}_i^t\|,$$

compressed by the learned MLP $\mathbf{e}_i^t = E_{\text{dyn}}(\mathbf{g}_i^t)$, $E_{\text{dyn}} : 12 \rightarrow 16 \rightarrow 16 \rightarrow d_s$ (default $d_s=4$).

3 Koopman propagation

A single linear operator $K \in \mathbb{R}^{d \times d}$ advances the observable, and the horizon is produced by iterating it:

$$\psi_i^{t+1} = K \psi_i^t, \quad \hat{\mathbf{p}}_i^{t_0+\tau} = C K^\tau \psi_i^{t_0}, \quad \tau = 1, \dots, T_p.$$

The social block is propagated by K as part of ψ (neighbours' futures are not recomputed during rollout).

4 Operator via Extended DMD

Stack the lifted training transitions column-wise, $\Psi = [\psi^t]$, $\Psi' = [\psi^{t+1}]$ over all consecutive pairs (all agents, all training scenes). EDMD gives K in closed form,

$$K = \Psi' \Psi^\dagger = \Psi' \Psi^\top (\Psi \Psi^\top + \lambda I)^{-1},$$

with a small ridge λ . The solve is differentiable in Ψ , hence in the encoder parameters.

5 Training objective (v1: plain consistency)

The only learnable parameters are the social encoder $\theta = \text{params}(E_{\text{dyn}})$, trained to minimise the one-step Koopman consistency residual:

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{P}|} \sum_{(t) \in \mathcal{P}} \|\psi^{t+1}(\theta) - K \psi^t(\theta)\|_2^2, \quad K = K(\theta) \text{ via EDMD.}$$

Since K is the EDMD minimiser for the current encoding, $\mathcal{L}$ is the minimal linear-fit residual; minimising it over θ pushes the lifted coordinates toward a Koopman-invariant (linearly evolvable) subspace (*cf.* LKIS, Takeishi et al. 2017). *Caveat:* a plain consistency loss admits a trivial collapse of the social block (a constant embedding is perfectly self-predictable); this v1 accepts the risk and we compare against alternatives (e.g. a state-block multi-step prediction loss) if needed.

Procedure. per minibatch (here: full batch): encode $\rightarrow \Psi, \Psi'$; solve EDMD for K ; compute $\mathcal{L}$; backprop to θ . After training, refit one global K on all training pairs. One model per held-out scene.

6 Adaptive online operator (optional)

At inference an agent-specific operator can track the agent's recent dynamics by a streaming rank-one update over *observed* lifted states,

$$K_i \leftarrow K_i + \eta \frac{(\psi^{s+1} - K_i \psi^s) \psi^{s\top}}{\|\psi^s\|^2},$$

applied for each consecutive observed pair (ψ^s, ψ^{s+1}) before the forward rollout ($\eta=0$ recovers the static global K). The update must use *real* observations: applying it to the predicted rollout gives a zero residual (no-op), which is exactly why the legacy implementation's adaptation was inert. With the time-delay-8 observable and the standard 8-step protocol only one observable is available, so adaptation is effective only with a longer / streaming history; the mechanism is implemented (`eta`) and left off in the v1 benchmark.

7 Relation to predecessors

- **KoopCast**: Koopman + MDN decoder, k -nearest relative neighbours. (removed from the repo.)
- **KoopyAdapt**: adds an obstacle branch, sector social, online adaptation; uses encoders + a learned decoder.
- **KoopCast++ (this model)**: *no decoder* (state in the observable), *no obstacle branch*, social-only learned dictionary, EDMD operator, plain consistency loss, $R = 2$ m.

8 v1 results (ETH/UCY leave-one-out)

Same neighbour-aware target set per scene (`evaluate_scene`); ADE/FDE in metres, $8 \rightarrow 12$. Baseline is constant velocity (last observed velocity extrapolated).

scene	ConstantVelocity		KoopCast++ (static)	
	ADE	FDE	ADE	FDE
eth	1.076	2.282	1.066	2.186
hotel	0.319	0.614	0.499	1.087
univ	0.604	1.339	0.724	1.523
zara1	0.427	0.952	0.436	0.971
zara2	0.322	0.724	0.354	0.788
AVG	0.550	1.182	0.616	1.311

Reading. The v1 model (plain consistency loss) is on par with constant velocity on ETH (slightly better) and ZARA1, but worse on HOTEL/UNIV/ZARA2, so on average it does not yet beat the baseline. The model is well-behaved (operator spectral radius ≈ 1 , no rollout blow-up), unlike the old KoopCast inference path. This is the expected v1 checkpoint; next iterations vary the training loss (state-block multi-step), social representation, and d_s , comparing back to this table.

9 Defaults / open knobs

- social embedding width d_s (default 4); E_{dyn} MLP size.
- EDMD ridge λ (default 10^{-4}); minibatch vs. global fit.
- alternative losses (state-block multi-step prediction) for the next iteration.
- online K adaptation (η) for streaming / longer-context use.